

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Институт информационных технологий
Кафедра информационных систем и технологий

ОТЧЕТ
по лабораторной работе № 2

Вариант № 23

по дисциплине «Системное программирование»

Выполнил:
студент группы 381574
специальности ПОИТ

Сушко Алексей Юльевич

Проверил:
ассистент кафедры ИСиТ,

Павлюченко К.А.

Минск, 2025

Тема работы: Взаимодействие процессов

Цель работы: Научиться осуществлять передачу произвольных данных между параллельно выполняющимися процессами.

Выполнение работы

Необходимо создать два процесса: клиентский и серверный. Серверный

процесс ждет ввода пользователем текстовой строки и по нажатию клавиши Enter инициируются следующие действия:

- клиентский процесс ожидает уведомления о том, что серверный процесс готов начать передачу данных (синхронизация);
- серверный процесс передает полученную от пользователя строку клиентскому процессу, используя либо каналы, либо сегменты разделяемой памяти / файловые проекции;
- клиентский процесс принимает строку и выводит ее на экран;
- серверный процесс ожидает уведомления от клиентского процесса об успешном получении строки;
- серверный процесс вновь ожидает ввода строки пользователем и т.д.\

В данной работе продолжается освоение синхронизации процессов. Уведомление процессов должно производиться посредством семафоров. Реализация механизма непосредственной передачи данных остается на выбор студента, однако в теории освоены должны быть все варианты.

Листинг кода программы:

ServerSide Program.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.IO.Pipes;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace ServerSide
{
    class Server
    {
        static Semaphore sem = new Semaphore(2, 2);
        public Server() { }
        public void ServerIPX()
        {
            NamedPipeServerStream sps = new NamedPipeServerStream("char",
                PipeDirection.InOut, 3);

            sps.WaitForConnection();
            sem.WaitOne();

            Console.ForegroundColor = ConsoleColor.Cyan;
            Console.WriteLine("\nКо мне подключился клиент. Мой поток: " +
                Thread.CurrentThread.ManagedThreadId);
            Console.ForegroundColor = ConsoleColor.Magenta;
```

```

        StreamWriter sw = new StreamWriter(sps);
        StreamReader sr = new StreamReader(sps);
        sw.AutoFlush = true;

        while(true)
        {
            Console.WriteLine(sr.ReadLine());
            Console.WriteLine("Выберите пункт меню");
            Console.WriteLine("1. Отправить на клиент строку\n2. Отключить
клиента\n");
            switch (Console.ReadKey().Key)
            {
                case ConsoleKey.D1:
                    Console.WriteLine("\nВведите строку, которая будет отправлена на
клиент: ");
                    sw.WriteLine(Console.ReadLine());
                    break;
                case ConsoleKey.D2:
                    Console.WriteLine("\n" + sem.Release());
                    break;
            }
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.ForegroundColor = ConsoleColor.DarkMagenta;

        Server objServ = new Server();
        Thread[] mas = new Thread[3];
        for (int i = 0; i < mas.Length; i++)
        {
            mas[i] = new Thread(objServ.ServerIPX);
            mas[i].Start();
            Console.WriteLine("Запущен сервер в потоке: " +
mas[i].ManagedThreadId);
        }
    }
}

```

ClientSide Program.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.IO.Pipes;
using System.IO;
using System.Diagnostics;

namespace ClientSide
{
    class Worker
    {
        public Worker() { }
        public void Client()
        {
            NamedPipeClientStream clp = new NamedPipeClientStream(".", "char",
PipeDirection.InOut);
            clp.Connect();
            Console.ForegroundColor = ConsoleColor.DarkBlue;
            Console.WriteLine("Я подключился к серверу.");
        }
    }
}

```

```

StreamWriter sw = new StreamWriter(clp);
StreamReader sr = new StreamReader(clp);
sw.AutoFlush = true;
sw.WriteLine($"Клиент с процессом: {Process.GetCurrentProcess().Id}");

while (true)
{
    Console.WriteLine(sr.ReadLine());
    Console.WriteLine("Введите строку, которая будет отправлена на сервер:");

    sw.WriteLine(Console.ReadLine());
}

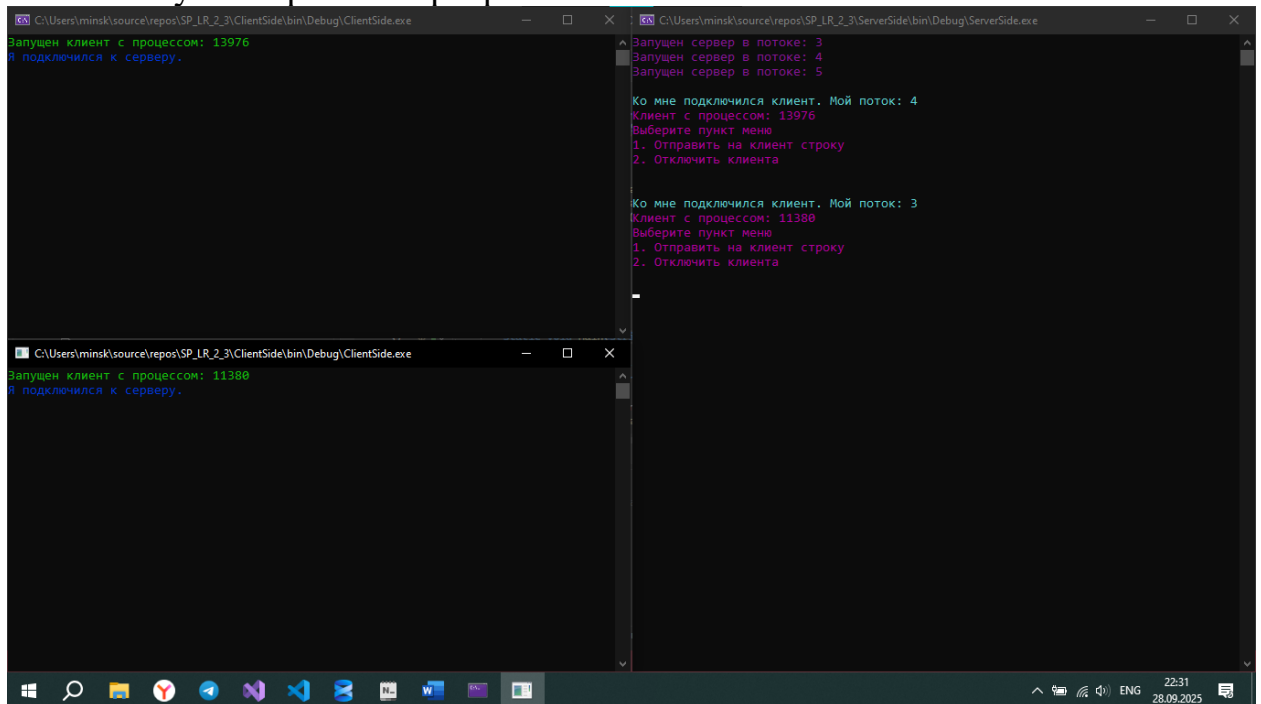
}

}

class Program
{
    static void Main(string[] args)
    {
        Worker objWorker = new Worker();
        Thread objTrh = new Thread(objWorker.Client);
        objTrh.Start();
        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"Запущен клиент с процессом: {Process.GetCurrentProcess().Id}");
    }
}

```

Результат работы программы:



Выводы

В данной лабораторной работе я изучил передачу произвольных данных между параллельно выполняющимися процессами.